

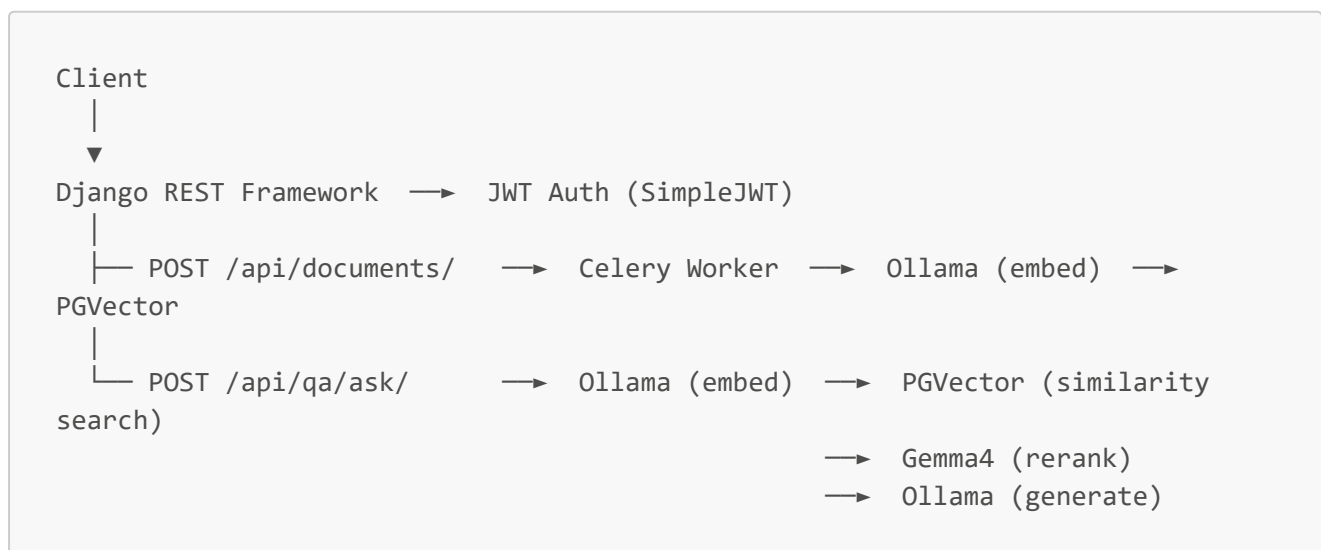
DocRAG — Document Question-Answering System

A POC (Proof-Of-Concept) for production-ready RAG (Retrieval-Augmented Generation) backend built with Django REST Framework, PGVector, Celery, and Ollama. Upload documents, generate embeddings asynchronously, and ask natural-language questions against your own document library.

Table of Contents

- [Architecture](#)
 - [Tech Stack](#)
 - [Project Structure](#)
 - [Getting Started](#)
 - [Prerequisites](#)
 - [Environment Variables](#)
 - [Running with Docker Compose](#)
 - [Running Locally \(without Docker\)](#)
 - [API Reference](#)
 - [Auth](#)
 - [Documents](#)
 - [QA](#)
 - [Testing](#)
 - [Admin Panel](#)
-

Architecture



Document processing is fully asynchronous. After upload the API returns immediately while a Celery worker chunks the file and writes embeddings into PGVector. The `/status/` endpoint lets clients poll until processing is **completed**.

Tech Stack

Layer	Technology
API framework	Django 4.x + Django REST Framework
Auth	SimpleJWT (access + refresh tokens, blacklisting)
Database	PostgreSQL with pgvector extension
Vector store	LangChain langchain-postgres (PGVector)
Embeddings / LLM	Ollama (local models — embeddinggemma , gemma4:e4b)
Task queue	Celery + Redis
Filtering / search	django-filters
API schema	drf-spectacular (Swagger UI + ReDoc)
Testing	pytest + pytest-django

Project Structure

```
backend/
├── core/                                # Django project config
│   ├── settings.py
│   ├── urls.py
│   └── celery.py
├── accounts/                           # Custom User model, JWT auth views
│   ├── models.py
│   ├── serializers.py
│   ├── views.py
│   └── urls.py
├── documents/                          # Document upload, chunking, embedding pipeline
│   ├── models.py
│   ├── serializers.py
│   ├── views.py
│   ├── tasks.py                       # Celery task: process_document_embedding
│   └── utils.py                       # PGVector connection helper
├── qa/                                 # Question answering, streaming, activity log
│   ├── models.py                     # QuestionActivity
│   ├── views.py                      # Ask / Stream / Activity endpoints
│   ├── reranking.py                  # Gemma4 reranker
│   └── streaming.py                  # StreamOptimizer
└── tests/
    ├── conftest.py
    ├── test_accounts.py
    ├── test_documents.py
    ├── test_qa.py
    ├── test_integration.py
    └── test_workflow.py
```

Getting Started

Prerequisites

- Docker & Docker Compose, **or** Python 3.11+, PostgreSQL, Redis
- [Ollama](#) with the following models pulled:

```
ollama pull embeddinggemma
ollama pull gemma4:e4b
```

Environment Variables

Copy `.env.example` to `.env` and fill in the values:

```
SECRET_KEY=your-secret-key-here
DEBUG=True
ALLOWED_HOSTS=localhost,127.0.0.1

# PostgreSQL
DATABASE_URL=postgresql://raguser:ragpassword@localhost:5432/ragdb

# Redis / Celery
REDIS_URL=redis://localhost:6379/0

# Ollama
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_EMBED_MODEL=nomic-embed-text
OLLAMA_CHAT_MODEL=gemma4:e4b

# JWT lifetime
ACCESS_TOKEN_LIFETIME_MINUTES=60
REFRESH_TOKEN_LIFETIME_DAYS=7
```

Running with Docker Compose

```
docker-compose up --build
```

This starts PostgreSQL (with pgvector), Redis, the Django API, and the Celery worker. The API is available at <http://localhost:8000>.

Apply migrations on first run:

```
docker-compose exec web python manage.py migrate
docker-compose exec web python manage.py createsuperuser
```

Running Locally (without Docker)

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Apply migrations
python manage.py migrate

# 3. Start the API
python manage.py runserver

# 4. Start Celery worker (separate terminal)
celery -A core worker -l info -Q documents,qa

# 5. (Optional) Start Celery Beat for scheduled tasks
celery -A core beat -l info
```

API Reference

Interactive Swagger UI is available at <http://localhost:8000/api/docs/> when `DEBUG=True`.

All endpoints except `register` and `login` require the header:

```
Authorization: Bearer <access_token>
```

Auth

Method	Endpoint	Description
POST	/api/auth/register/	Create a new user account
POST	/api/auth/login/	Login — returns <code>access</code> + <code>refresh</code> tokens
POST	/api/auth/refresh/	Rotate access token using refresh token
POST	/api/auth/logout/	Blacklist refresh token
GET	/api/auth/me/	Get current user profile
PATCH	/api/auth/me/	Update current user profile

Register request body:

```
{
  "username": "alice",
  "email": "alice@example.com",
  "password": "StrongPass123!",
  "password_confirm": "StrongPass123!"
}
```

Login response:

```
{
  "access": "<jwt>",
  "refresh": "<jwt>",
  "user": { "id": "uuid", "username": "alice", "email": "...",
  "question_count": 0 }
}
```

Documents

Method	Endpoint	Description
POST	/api/documents/	Upload a document (multipart/form-data, field: file)
GET	/api/documents/	List own documents (paginated, filterable)
GET	/api/documents/{id}/	Get document detail
DELETE	/api/documents/{id}/	Delete a document
GET	/api/documents/{id}/status/	Poll processing status
GET	/api/documents/{id}/chunks/	Debug: view stored vector chunks

Supported file types: txt, pdf, xlsx, xls, csv

List query parameters:

Param	Type	Description
status	string	Filter by pending, processing, completed, failed
file_type	string	Filter by extension
search	string	Search by filename
ordering	string	e.g. -created_at, file_name
page	int	Page number

Document status flow:

```
pending → processing → completed
      ↘ failed
```

QA

Method	Endpoint	Description
POST	/api/qa/ask/	Synchronous question answering
POST	/api/qa/stream/	Streaming question answering (NDJSON)
GET	/api/qa/activity/	Paginated question history + stats

Ask request body:

```
{
  "question": "What is retrieval-augmented generation?",
  "document_id": "uuid",
  "page": 1,
  "chunk_index": 0
}
```

Only **question** is required. **document_id**, **page**, and **chunk_index** narrow the vector search.

Ask response:

```
{
  "answer": "RAG combines retrieval systems with generative models...",
  "sources": [
    {
      "file_name": "report.pdf",
      "page": 1,
      "chunk_index": 2,
      "excerpt": "RAG combines retrieval..."
    }
  ],
  "response_time_ms": 842,
  "status": "success"
}
```

Stream endpoint returns **application/x-ndjson** — one token per line:

```
{"token": "RAG "}
{"token": "combines "}
```

```
{"token": "retrieval..."}
```

Activity history query parameters:

Param	Type	Description
status	string	success, no_answer, error
document_id	uuid	Filter by document
page	int	Page number
page_size	int	Items per page (default 20)

Testing

```
# Run the full test suite
pytest

# Run a specific domain
pytest tests/test_documents.py -v
pytest tests/test_qa.py -v
pytest tests/test_integration.py -v -m integration

# With coverage
pytest --cov=. --cov-report=html
# Open htmlcov/index.html in a browser
```

Test markers defined in `pytest.ini`:

Marker	Description
accounts	Auth and user management tests
documents	Document upload and processing tests
qa	Question answering tests
integration	End-to-end workflow tests

External dependencies (Ollama, PGVector) are mocked in all tests. No live services are required to run the test suite.

Admin Panel

The Django admin is available at <http://localhost:8000/admin/>.

Model	Features
-------	----------

Model	Features
User	View question count, manage permissions, search by username/email
Document	Filter by status and file type, bulk mark as pending/failed, colour-coded status badges
QuestionActivity	Read-only audit log, filter by status, sources count column, no add permission (system-generated only)